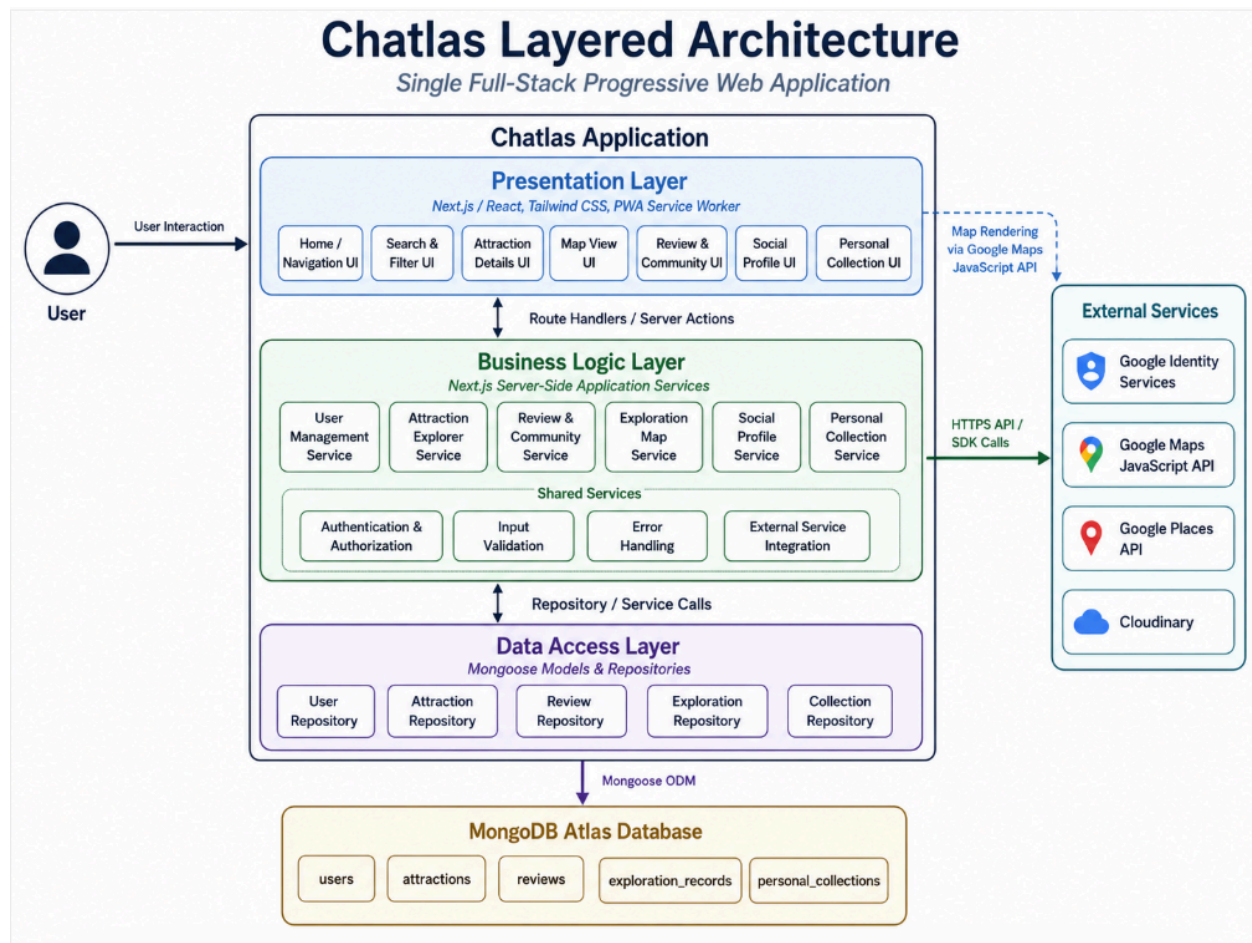


Internal Software Structure / Architecture



Brief Description of Chatlas Layered Architecture

Chatlas adopts a Layered Architecture to organise the internal software structure into three logical layers: the Presentation Layer, Business Logic Layer, and Data Access Layer. All layers are implemented within a single full-stack Progressive Web Application project to reduce development and integration complexity.

The Presentation Layer is responsible for user interaction and visual output. It is developed using Next.js, React, Tailwind CSS, and PWA technologies. This layer contains the interfaces for attraction searching, map viewing, reviews, social profiles, and personal collections. The PWA Service Worker also supports limited offline access to previously cached content.

The Business Logic Layer handles the main rules and processing of the system. It contains the six Chatlas modules: User Management, Attraction Explorer, Review & Community, Exploration Map, Social Profile, and Personal Collection. It is also responsible for authentication, authorisation, input validation, error handling, exploration progress calculation, and external-service integration. These modules are consistent with the confirmed Chatlas project scope.

The Data Access Layer manages communication between the business logic and MongoDB Atlas. It uses Mongoose models and repositories to perform data retrieval, creation, updating, and deletion. This layer stores and manages information such as users, attractions, reviews, exploration records, and personal collections.

The layers communicate through clear internal interfaces. The Presentation Layer sends requests to the Business Logic Layer through Route Handlers or Server Actions. The Business Logic Layer accesses stored data through repository or service calls, while the Data Access Layer communicates with MongoDB through Mongoose ODM.

Chatlas also integrates external services through HTTPS APIs or SDK calls. Google Identity Services supports social login, Google Maps JavaScript API displays interactive maps, Google Places API provides attraction information, and Cloudinary stores uploaded images. This architecture improves maintainability, keeps responsibilities clearly separated, and allows the team to develop the system within one shared project.